Team # 27
M1: Approach — Accessible Shopping Layer

Megathon 2026 • Bhashini track

1. Problem Statement and Our Understanding

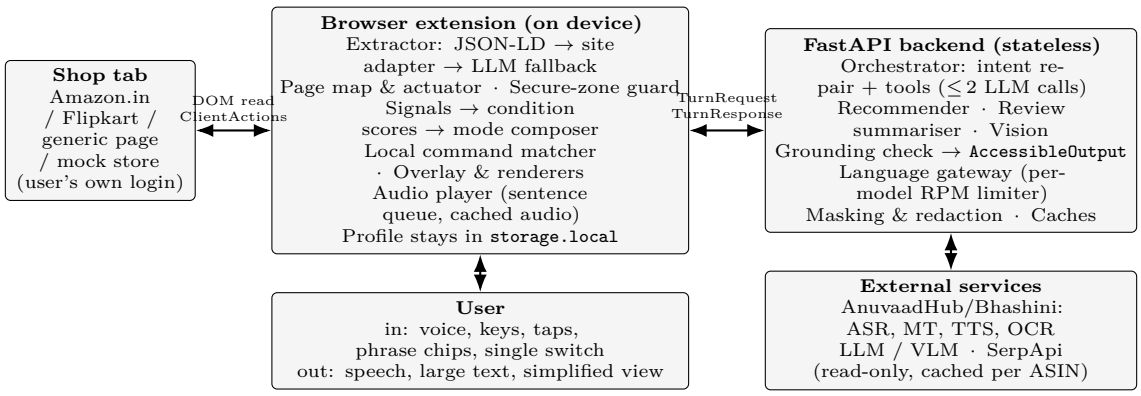
Indian e-commerce sites (Amazon.in, Flipkart and others) are built for sighted users with steady hands, clear speech and good English. People with **visual**, **speech** (including dysarthria), **hand-motor** or **age-related** impairments, and people who prefer an Indian language, have trouble searching, comparing, understanding reviews and checking out. Today's tools only help in parts: screen readers read cluttered pages line by line, voice assistants fail on unclear speech, and translation tools mistranslate prices and brand names. We see four constraints that any solution must meet:

- **It must work on the real sites people already use**, in their own logged-in browser, without the shop changing anything.
- **Trust:** product facts (price, specs, ratings) must never be invented, and money must never move without explicit confirmation.
- **Conditions overlap**, e.g. an older blind user with a hand tremor, so we cannot ship one generic "accessibility mode".
- **Language:** English plus Hindi, Telugu and other Indian languages, with brand names, units and prices kept intact.

2. Idea / Abstract

We are building a **browser extension** (Chrome desktop and Firefox for Android) and a **FastAPI backend** that together add an accessible layer on top of any shopping page. The extension pulls structured product data out of the live page (`ProductJSON`) and builds a map of the actions available on it. The backend, using Bhashini/AnuvaadHub ASR, MT, TTS and OCR plus an LLM, turns each user request into an `AccessibleOutput`: a short spoken summary, structured sections, a plain-language version, and a *grounding* entry that links every claim to a source field. Users control everything by tolerant voice, keyboard, tap, phrase chips or single-switch scanning. Output is sentence-queued speech with full playback controls, large text, or a simplified view that we render ourselves; we never restyle the shop's page. Condition modes are *inferred on device* from passive interaction signals. A mode is only *suggested* (at least two independent signals, a 10 s announced countdown, always undoable), and the user can combine several modes freely. Checkout always stops at a spoken and visual order summary. Payment details are entered only by the user, inside a "secure zone" where the microphone is muted and nothing is extracted.

3. High-Level Architecture and Pipeline



Shared contracts. JSON Schemas (`ProductJSON`, `PageContext`, `Profile`, `TurnRequest/Response`, `AccessibleOutput`, `Confirmation`) generate both the TypeScript and the Pydantic types, so four parallel tracks (Speech, Page, Brain, Access UX) integrate against mocks from day one.

Product open (precompute in parallel, speak one thing at a time). (1) Extraction starts at page load; title and price are spoken right away from meta tags. (2) Payable fields (price, stock, delivery, seller) always come from the live page. SerpApi optionally adds specs, review insights and similar items, from one cached call per ASIN. (3) The specs, reviews (pros/cons/verdict with mention counts), image descriptions (OCR first, VLM for the rest) and comparison sections are computed concurrently and cached. (4) Only the section the user asks for is translated and synthesised.

One voice turn (target < 4 s end to end). (1) Patient capture: 2.5 s silence timeout in speech mode, and the user is never cut off. (2) A local fuzzy matcher handles commands and on-screen labels without a server call. (3) Bhashini ASR runs in the chosen language. (4) LLM intent repair returns a confidence: $\geq$0.8 acts; 0.5–0.8 asks "Did you mean X or Y?"; $<$0.5, or two failed tries, offers phrase chips. (5) Tools run as plain functions; code ranks comparison candidates and the LLM only explains the ranking. (6) Every claim is grounding-checked. Brand names, units and prices are masked before MT and restored after. (7) TTS streams sentence by sentence through the player (pause, repeat, back, next section, slower, faster, all from one key or by voice).

Accessibility per condition, composed by rules. *Visual:* voice-first overlay that works with screen readers, spoken overview, ordered summaries. *Speech:* the unclear-speech layers above, phrase chips, word prediction. *Motor:* action menu with large targets, two-level switch scanning, no timeouts. *Ageing:* simplified view, step-by-step checkout wizard, large text, "explain simpler". *Colour (manual only):* colour names on every swatch. The composer merges the active conditions into one mode: it takes the most restrictive pacing, picks output channels per condition, and uses an input fallback ladder (voice → keys/tap → chips → switch) that always ends at a single binary input.

Safety and privacy. Consequential actions (cart, quantity, address, order, return) return a `Confirmation`; the LLM can never press "Place order". Card numbers, OTPs, Aadhaar, PAN, phone numbers and addresses are redacted to placeholders before any text reaches the LLM or MT. The backend never sees shop cookies or credentials. All language-model tokens stay in one backend gateway.

4. Planned Testing Methodology

Area	Method and metric
Contracts & CI	Every fixture is validated against the JSON Schemas; unit tests (pytest, TS) and lint run in `make check` on each PR; `MOCK=1` recorded responses for the language APIs and SerpApi.
Extraction	Field-level precision/recall of `ProductJSON` on saved Amazon.in, Flipkart and generic pages, checked against hand labels; missing fields must appear in `missing`, never be guessed.
Grounding	Automatic check that every claim in `AccessibleOutput` maps to a real field path or fetched review text; target: 0 ungrounded claims on the test set.
Unclear speech	TORGO dysarthric samples on a fixed command set: command accuracy for plain ASR vs. our full stack (local matcher + LLM repair + disambiguation). English only; simulated testing, not user validation.
Condition inference	Recorded sessions of normal use and of each simulated condition; weights tuned until normal sessions trigger no suggestions. We report the false-suggestion rate and the time to a correct suggestion.
Multilingual	Hindi/Telugu/Hinglish test phrases with numbers, prices and brand names: the masking round-trip must preserve every masked token exactly; manual adequacy check of MT and TTS.
Safety	Scenario tests on the mock store: no order without explicit confirmation; mic muted and nothing extracted in payment/OTP fields; redaction catches Luhn-valid cards, OTPs, Aadhaar and PAN.
Accessibility	axe-core audit of the overlay; manual runs with NVDA/ChromeVox (desktop) and TalkBack (Android); 44 px targets, keyboard and switch reachability, reduced motion.
Personas & latency	End-to-end task scripts (search → compare → cart → checkout summary) per condition and for the required combinations (visual+motor, visual+speech, speech+motor, ageing+any); per-turn latency logged against the 4 s budget.